* palindrome checking

S = "ROTOR"

S == S[::-1] True ✓

Day - 6 mar - 02 Python slicing

Python Slicing: (Very Easy)

→ str-obj[start : stop : step] positions to jump

↳ included ↳ ends (included)

→ S[:] stop

↑ Default; 0 ↳ Default value; len(sequence)

→ S = "Phanindra"

↳ S[:] → S[0:9]

-9	-8	-7	-6	-5	-4	-3	-2	-1
p	h	a	n	i	n	d	r	a
0	1	2	3	4	5	6	7	8

S[-4:-1] → 'ndr'

↓

-4 to (-1-1)

-4 to -2

* S[-4:] → Default step is +1

↳ convert the negative indexes -4 ⟺ 5

-7 ⟺ 2

S[5:2] → (Start < End) false.

So ' ' Empty space.

S[-4:-7:-1] (Start > End) ✓

↓ 'nin' ↓ [5:2:-1] [5→4→3]

↓ Not included

-9	-8	-7	-6	-5	-4	-3	-2	-1
p	h	a	r	i	n	d	r	a
0	1	2	3	4	5	6	7	8

* "Pharindra" [-7:-4:-1]
  ↓
  convert negative indexes become [2:5:-1]
  ↓
  (start > end) false so ' '

If step = +1 (+ve) → condition start < stop
step = -1 (-ve) → " start > stop

* Default value when step is Negative:

V. Imp If 'step' is -ve & start is missing, then
start = -1 (last character) [start from last character]

* ~~step~~
  S [:-7:-1]
  ↓
  S[-1:-7:-1]
  ↓
  S[8:2:-1] ← ~~[8→7→6→5→4→3]~~
  ↓ "ardnin"

  8 to (2+1)
  8 to 3 ✓

* If step is -ve, End is missing, then

Stop = None (but logically before index 0)

Stop = -len(s)-1 (conceptually before 0) (Stop before first index)

s[-4: : -1] → s[-4:-10:-1] "

-10 → very left boundary

s[5:-10:-1] → n in app

Stop before index 0

[5 to 0] inverse.

*

* s[4:-3:-1]

s[4:6:-1] → ' '

(4>6) false

* When (step>0):

missing part	Default value
start	0
Stop	len(sequence)
Step	(1)

* When (step<0)

missing part	Default value
start	len(seq)-1 or -1
Stop	before index 0

If End<0 [then -1]
End ≥ 0 [then len(seq)-1]
both are same only

-9	-8	-7	-6	-5	-4	-3	-2	-1
p	h	a	n	i	n	d	r	a
0	1	2	3	4	5	6	7	8

* S = "Phanindra"

→ S[2:8:-1] step -ve; (begin > end) false so ''

* S[-1:-10:-3] → Entire string reverse with step -3

→ -1 to (-10+1)

(-1 to -9) 'a n a'

before index '0'

* S[100:2:-1] Start adjusts to 8

"a r d n i n"

becomes S[8:2:-1]

8 to 3

* S[-100:3] Start adjusts to 0

* S[3:-100:-1]

S[3:-100:-1] → 'n a h p'

to the end of string in reverse

before index 0

* S[-3:100]

S[6:100]

S[6:9]

6 to 8 → 'd r a'

* s[5: : -2]

s[5: → : -2]

before index 0

'h nh'

* s[-4: -1: -1]

s[5: 8: -1]

(5 > 8) false so ''

* s[4: -3: -1] (4 → 6) invalid direction

s[4: 6: -1]

(start > end) false; so ''

* s[-3: 4: -1]

s[6: 4: -1] → 'dn'

→ s[1: -8: -1]

s[1: 1: -1] → ''

both equal (start > end false) so ''

Lec 6 - Day06 ↔ .ipynb [code here]

Lec-19 - ~~lists~~ ↔ Bytes Data Type & bytearray

↔ 2. bytes Data Type [sequence data Type]

→ 'bytes' is one of the pre-defined class

→ The purpose of bytes data type is that "To implement End-to-End" in network based applications.

→ To implement End-to-End Encryption in n/w based applications, bytes data type organizes the numerical integer values in range of 0 to 256 i,e it stores 0 to 255 values only.

→ bytes data type does not contain any symbolic notation for storing bytes data because programmer will not store bytes data directly in the program. But we can convert any other type of values into bytes type by using bytes().

Syntax: varname = bytes(object)

→ Ex:

→ lst = [100, 200, 10, 45] # list type

b = bytes(lst) → gives result in format where humans cannot understand it

print(b)

b'd\xc8\n-' → End to End Encryption Humans unreadable format

This normal data converted to humans unreadable format for Encryption (for Data security)

* An object of bytes maintains Insertion order (Insertion order is nothing but whatever the order we organize in the same order, the data will be displayed)

→ on the object of bytes, we can perform both indexing and slicing operations

→ bytes is immutable object, because bytes object does not support item assignment.

*

Bytearray:

→ Bytearray is a mutable object, [support item assignment]

→ The functionality of bytearray is exactly similar to bytes, but the bytes objects are immutable because bytes object does not support item assignment.